

Introduction to C Programming

Gabby Holden 17/10/2023

Basic Syntax in C

Basic Structure of a C Program

```
#include <stdio.h> //include brings in external libraries

int main(){ //main() is the entry point of the program
    printf("Hello World!\n");
    return 0; //return 0; informs the operating system the program was terminated successfully
}
```

Variables and Data Types

Variables are used to store and manipulate data in a program and have a data type, a name, and a value.

```
// Data types
int age = 15; // int is used for integers (i.e. positive or negative whole numbers)
char grade = 'A' // char is used to store a single character or small integers (ASCII values)
float price = 19.99; // float is used for numbers with decimal points (with limited precision)
double pi = 3.14159265359 // similar to float, but provides higher precision for decimal numbers
```

Unlike other programming languages, notice C does not have a built in 'string' variable: instead, C uses arrays of characters to represent strings.

```
char myString[] = "Hello World!";
```

Loops

for loops execute a block of code for a specified number of times

```
// general structure:
for (initialization; condition; increment) {
    // code to be executed
}

// a more specific example:
for (int i = 0; i < 5; i++) { // Initialize i to 0; Execute while i is less than 5; Increment i after each iteration
    printf("Iteration %d\n", i); // Print the current value of i
}
```

Loops

while loops execute a block of code as long as a specified condition is true

```
// general structure
while (condition) {
    // code to be executed
}

// a more specific example:
int i = 0; // Initialize the loop control variable
while (i < 5) { // Continue while i is less than 5
    printf("Iteration %d\n", i); // Print the current value of i
    i++; // Increment i after each iteration
}
```

Loops

do while loops execute a block of code as least once and then repeatedly as long as a specified condition is true e.g.

```
// general structure
do {
    // code to be executed
} while (condition);

// a more specific example:
int i = 0; // Initialize the loop control variable
do {
    printf("Iteration %d\n", i); // Print the current value of i
    i++; // Increment i after each iteration
} while (i < 5); // Continue while i is less than 5
```

Conditional Statements

if statements execute a block of code if a specified condition is true

else statements are used with an if statement and execute a block of code if the condition is false

```
// if statement
//general structure
if (condition) {
    // code to execute if the condition is true
} else {
    // code to execute if the condition is false
}

// a more specific example:
int num = 10; // You can replace this with any number, but you need to define num before referring to it in the if statement
if (num % 2 == 0) { // Check if the number is even
    printf("%d is even.\n", num); // Code to execute if it's even (printing that the value is even)
} else {
    printf("%d is odd.\n", num); // Code to execute if it's odd (printing that the value is odd)
}
```


Conditional Statements

switch statements select one of many code blocks to be executed e.g.

```
// general structure
switch (expression) {
    case value1:
        // code to execute if expression matches value1
        break;
    case value2:
        // code to execute if expression matches value2
        break;
    // more cases...
    default:
        // code to execute if expression doesn't match any case
}
```

```
// a more specific example:
int day = 3; // Replace with any value from 1 to 7 (1 = Sunday, 7 = Saturday)

switch (day) {
    case 1:
        printf("Sunday\n");
        break;
    case 2:
        printf("Monday\n");
        break;
    case 3:
        printf("Tuesday\n");
        break;
    case 4:
        printf("Wednesday\n");
        break;
    case 5:
        printf("Thursday\n");
        break;
    case 6:
        printf("Friday\n");
        break;
    case 7:
        printf("Saturday\n");
        break;
    default:
        printf("Invalid day\n");
}
```

Functions

Must be declared before use, consists of:

Function name – name of the function (should be a valid C identifier)

Return type – the data type of the value that the function returns. If the function doesn't return value you use 'void'.

Parameters – variables or values that the function accepts as an input. If the function doesn't take any parameters, use empty parentheses ().

Return value – line used to return a value from the function. This value must match the return type!

Functions

```
// general example
return_type function_name(parameters) {
    // Function body: code to perform the task
    // ...
    return value; // Return a value of the specified return_type
}
```

```
// a more specific example
#include <stdio.h>
// 'add' function to calculate the sum of two integers
int add(int a, int b) {
    int result = a + b;
    return result;
}
// 'main' function
int main() {
    int num1 = 5;
    int num2 = 7;
    int sum = add(num1, num2); // call the add function

    printf("The sum of %d and %d is %d\n", num1, num2, sum);
    return 0;
}
```

Pointers

A **pointer** is a variable that stores the memory address of another variable.

The use of pointers are a *fundamental part of c programming*

Commonly used for tasks like dynamic memory allocation, working with arrays, and building data structures

They provide low-level control over memory, which can be both powerful and error-prone, so it's important to use them carefully.

Pointers

```
// general example
data_type *pointer_name; // Declaration of a pointer
// We then use the address-of operator & to assign the memory

// a more specific example
int main() {
    int number = 42; // Declare an integer variable
    int *pointer;    // Declare a pointer to an integer

    pointer = &number; // Assign the memory address of 'number' to the pointer

    printf("Value of number: %d\n", number);
    printf("Address of number: %p\n", &number);
    printf("Value of number using pointer: %d\n", *pointer);

    return 0;
}
```

Pointers

dynamic memory allocation functions (such as malloc and free) allocate memory during program execution rather than at compile time.

The **malloc** function takes a single argument (number of bytes to allocate) and returns a pointer to the first byte of the allocated memory block if the allocation is successful, else it returns a null pointer 'NULL'.

```
//general example for using malloc
int *array = (int *)malloc(5 * sizeof(int)); // Allocates memory for an array of 5 integers
if (array == NULL) {
    // Handle allocation failure
}
```

The **free** function is used to deallocate dynamic memory by taking a single argument (a pointer to the allocated memory), this then allows the memory to be returned and reused. To avoid accidentally modifying the freed memory, the pointer should be set to 'NULL'.

```
//general example for using free
free(array); // Deallocates the memory pointed to by 'array'
array = NULL; // Set the pointer to NULL after freeing the memory
```

Pointers

When would I want to use malloc and free?

Dynamic memory allocation is particularly useful when dealing with situations where the amount of data is not fixed in advance.

e.g. when:

- creating lists or arrays
- storing data of unknown size
- handling user input (of unknown size)
- when managing files, network connections or other resources that require memory to store information about them
- when implementing custom data structure where the size can vary (e.g. dynamic arrays, trees, linked lists)

```
// a more specific example using malloc and free
int main() {
    int *array; // Declare a pointer to an integer
    int size = 5; // Number of integers to allocate

    // Allocate memory for an array of integers
    array = (int *)malloc(size * sizeof(int));

    if (array == NULL) {
        printf("Memory allocation failed.\n");
        return 1; // Exit with an error code
    }

    // Initialize the array
    for (int i = 0; i < size; i++) {
        array[i] = i * 10;
    }

    // Print the array elements
    for (int i = 0; i < size; i++) {
        printf("array[%d] = %d\n", i, array[i]);
    }

    // Deallocate the allocated memory
    free(array);

    return 0;
}
```

Pointers

Can I program in C without pointers?

Yes. While pointers are a very useful feature of C, they are not always necessary, particularly with simple programs (e.g. those performing basic input/output operations). In these cases, you can write programs using just variables and functions.

But it's still important to learn how to use pointers :)

File Handling: Reading & Writing Files

```
int main() {  
    // Attempt to open the file "file.txt" in read mode ("r")  
    FILE *fp = fopen("file.txt", "r");  
  
    // Error handling: If fopen() returns NULL, it means the file could not be opened  
    // This can happen if the file doesn't exist or if there's a permission issue  
    if (fp == NULL) {  
        perror("Error opening file"); // Print the error message to stderr  
        exit(EXIT_FAILURE); // Exit the program if the file couldn't be opened  
    }  
  
    // File successfully opened, proceed with file operations (not shown here)  
    // Example: reading from the file with fscanf(), fgets(), etc.  
  
    // After performing the file operations, it's important to close the file  
    fclose(fp); // fclose() flushes and closes the file, freeing up system resources  
  
    return 0;  
}
```

Troubleshooting in C

Issues using the OpenSSL library?

Check that openssl is correctly installed:

go to the terminal and type:

`openssl version`

If you get an error, then it is not properly installed, and you will need to install it using your system's package manager or by downloading it from the official OpenSSL website.

Check your program is properly linked with openssl libraries:

open your terminal and navigate to the file that your c code is saved in (using the `cd` command). Then type the following for windows:

`gcc -o myprogram myprogram.c -lssl -lcrypto`

Else try: double check you path to source files, ensure you are using the correct compiler options, double check your code includes the correct OpenSSL header files

General issues with code errors?

Error messages typically provide information about what went wrong and where the issue occurred...when in doubt, **google the error** :) someone on the internet has probably had the same issue before!

Check for Typos!!!

Break your code into smaller units or functions. This can help isolate problems to specific areas and make it easier to identify the source of errors...also along these lines: avoid typing out your entire code and only then running when you've finished. It is better practice to **run in chunks** as you go along to help locate where and when mistakes are being made.

Ask for help: while we can't directly fix errors in your code or give answers to your assessment, we can hint at where you've gone wrong and point you in the direction of helpful resources!

Importantly...

USE COMMENTS IN YOUR CODE

Even if you are having issues resolving errors or your code doesn't run, you will still get marks for explaining what you are trying to do and for the decisions you made (e.g. whether your choices reflect strong security practices).

Questions?